



UNIVERSIDADE FEDERAL DE PERNAMBUCO
CAMPUS AGRESTE
NÚCLEO INTERDISCIPLINAR DE CIÊNCIAS EXATAS E DA NATUREZA
CURSO DE BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA

PEDRO LUCAS ALVES SILVA
IAGO FERREIRA SILVA

TRABALHO DE GRAFO

CARUARU
2026

UNIVERSIDADE FEDERAL DE PERNAMBUCO
CAMPUS AGRESTE
NÚCLEO INTERDISCIPLINAR DE CIÊNCIAS EXATAS E DA NATUREZA
CURSO DE BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA

RELATÓRIO

TRABALHO DE GRAFO

PEDRO LUCAS ALVES SILVA
IAGO FERREIRA SILVA

CARUARU

2026

RESUMO

Este trabalho apresentou a modelagem e a análise de uma rede de municípios de Pernambuco utilizando o dataset D8, representado por um grafo não direcionado e ponderado, em que os vértices correspondem às cidades e as arestas representam as conexões rodoviárias entre elas, com pesos equivalentes às distâncias em quilômetros. A implementação foi realizada em Python, utilizando listas de adjacência para representar o grafo, devido à sua eficiência para grafos esparsos. Foram implementados os principais algoritmos de grafos, incluindo Busca em Largura (BFS), Busca em Profundidade (DFS), algoritmo de Dijkstra e identificação de componentes conexas. Esses algoritmos permitiram analisar características importantes da rede, como conectividade, menor caminho entre cidades, identificação do município com maior número de conexões, detecção de ciclos e exploração da estrutura do grafo por meio das árvores de busca. A análise de complexidade mostrou que as soluções implementadas apresentam bom desempenho para a estrutura da rede, com complexidades compatíveis com grafos representados por listas de adjacência. Os resultados obtidos demonstram a importância da teoria dos grafos na representação e resolução de problemas relacionados a redes de transporte, evidenciando como esses algoritmos podem ser aplicados para compreender a organização e a conectividade entre os municípios de Pernambuco.

Palavras-chave: Grafos; Dijkstra; BFS; DFS; Lista de adjacência; Rede rodoviária.

1. Introdução

Este trabalho apresenta uma análise da rede de municípios de Pernambuco utilizando o dataset D8, representado por meio da teoria dos grafos. Nessa representação, cada município corresponde a um vértice e cada ligação rodoviária a uma aresta, cujo peso representa a distância em quilômetros entre as cidades. O grafo é não direcionado, ponderado e composto por 26 nós e 36 arestas.

A estrutura foi implementada utilizando lista de adjacência, por ser uma representação eficiente para esse tipo de rede. O objetivo do estudo é aplicar algoritmos clássicos de grafos, como BFS, DFS e Dijkstra, para analisar a conectividade da rede, determinar o menor caminho entre cidades, identificar o município com maior número de conexões, verificar a existência de ciclos e estudar as componentes conexas. Dessa forma, demonstra-se a aplicação da teoria dos grafos na análise de redes de transporte rodoviário.

2. Descrição da Implementação

A rede de municípios foi representada por uma lista de adjacência, implementada com um dicionário em Python, no qual cada município é um vértice e cada lista contém seus municípios vizinhos e as respectivas distâncias. Essa estrutura foi escolhida por ser mais eficiente para grafos esparsos, como o dataset D8, que possui 26 vértices e 36 arestas.

O grafo foi implementado como não direcionado, inserindo cada ligação nos dois sentidos, e ponderado, armazenando a distância entre os municípios como peso das arestas. Essa representação permite que algoritmos como o Dijkstra encontrem o menor caminho considerando a distância total percorrida.

O projeto foi dividido em três módulos: `estrutura_grafo.py`, responsável pela criação do grafo; `buscas_grafo.py`, que implementa os algoritmos de BFS, DFS, Dijkstra e demais análises; e `trabalho_grafos.py`, que integra os módulos e executa todas as análises da rede. Essa organização tornou o código mais modular e de fácil manutenção.

3. Análise de Complexidade

Nesta seção é apresentada a análise da complexidade dos algoritmos implementados no projeto. Como a representação escolhida foi a lista de adjacência, a eficiência dos algoritmos é expressa em função do número de vértices ($|V|$) e do número de arestas ($|A|$) da rede.

3.1. Busca em Largura (BFS)

A Busca em Largura (BFS) percorre o grafo visitando cada vértice uma única vez e explorando todas as suas arestas. Como cada vértice é inserido e removido da fila apenas uma vez e cada aresta é analisada uma única vez, a complexidade de tempo é $O(|V| + |A|)$.

Em relação ao espaço, o algoritmo utiliza estruturas auxiliares para armazenar os vértices visitados, a fila de processamento e o dicionário de pais, resultando em uma complexidade de $O(|V|)$.

3.2. Busca em Profundidade (DFS)

A Busca em Profundidade (DFS) também visita cada vértice apenas uma vez e percorre todas as arestas do grafo durante a execução. Dessa forma, sua complexidade de tempo é $O(|V| + |A|)$.

O algoritmo mantém conjuntos e dicionários para registrar os vértices visitados, tempos de descoberta, tempos de finalização e pais dos vértices. Além disso, utiliza a pilha de chamadas da recursão. Assim, sua complexidade de espaço é $O(|V|)$.

3.3. Algoritmo de Dijkstra

O algoritmo de Dijkstra calcula o menor caminho considerando os pesos das arestas (distâncias em quilômetros). A implementação utiliza uma fila de prioridade (heap), permitindo selecionar rapidamente o próximo vértice com menor distância conhecida.

Nessa implementação, cada inserção e remoção da fila possui custo $O(\log |V|)$, enquanto todas as arestas são analisadas ao longo da execução. Portanto, a complexidade de tempo é $O((|V| + |A|) \log |V|)$.

Em relação ao espaço, são armazenados os vetores de distâncias, pais e a fila de prioridade, totalizando $O(|V|)$.

3.4. Detecção de Componentes Conexas

A identificação das componentes conexas é realizada por meio de sucessivas buscas em largura (BFS). Cada vértice pertence a apenas uma componente e é visitado somente uma vez, enquanto cada aresta também é percorrida uma única vez.

Assim, a complexidade de tempo é $O(|V| + |A|)$. O algoritmo utiliza listas de vértices visitados, uma fila para a BFS e uma lista contendo as componentes encontradas, resultando em uma complexidade de espaço de $O(|V|)$.

Resumo das Complexidades

Algoritmo	Complexidade (Tempo)	Aplicação no Projeto
BFS	$O(V + A)$	Caminhos mínimos em número de saltos.
DFS	$O(V + A)$	Identificação de ciclos e tempos de visitação.
Dijkstra (com heap)	$O(A \log V)$	Menor rota rodoviária em km (Petrolina ↔ Recife).
Componentes Conexas	$O(V + A)$	Cálculo de grau médio e maior grau.

Como o dataset D8 possui 26 vértices e 36 arestas, trata-se de um grafo esparso, tornando a representação por lista de adjacência bastante eficiente. Nessa estrutura, todos os algoritmos implementados apresentam bom desempenho, pois percorrem

apenas os vértices e arestas realmente existentes na rede, evitando o processamento de posições vazias que existiriam em uma matriz de adjacência.

4. Respostas às Perguntas Obrigatórias

Q1 — Conectividade: o grafo é conexo? Quantas componentes existem?

A função `contar_componentes()` retornou apenas 1 componente, contendo as 26 cidades do dataset. Ou seja, o grafo é conexo: não existe nenhuma cidade nem grupo de cidades isolado do restante da rede. Isso é notável porque o número de arestas (36) está bem próximo do mínimo teórico necessário para conectar 26 vértices, que é 25 (uma árvore) — a malha tem pouca folga, mas nenhuma cidade fica de fora.

Q2 — Menor caminho entre Recife e Petrolina

Recife e Petrolina foram escolhidas por representarem os dois polos citados na introdução, em extremos opostos do estado. Como o objetivo é medir a distância real em quilômetros — e não apenas o número de rodovias percorridas —, o algoritmo utilizado foi o Dijkstra, que considera o peso das arestas; a BFS resolveria apenas o caminho com menos saltos, ignorando a distância.

Caminho encontrado: Recife → Caruaru → Arcoverde → Serra_Talhada → Salgueiro → Petrolina

Distância total: 710 km

O caminho passa por Caruaru logo na primeira aresta, o que reforça seu papel de elo entre o litoral e o interior identificado na introdução.

Q3 — Cidade com maior grau

A função `maior_grau()` aponta Recife como o vértice de maior grau, com 10 conexões diretas: Olinda, Paulista, Camaragibe, Jaboatão, Cabo de Santo Agostinho (Caboatã), Ipojuca, Vitória de Santo Antão, Caruaru, Limoeiro e Surubim. Esse resultado é coerente com a realidade: a capital concentra o porto, o aeroporto e o entroncamento das principais rodovias federais (como BR-101 e BR-232),

funcionando como hub logístico — a maior parte do tráfego de e para o interior do estado passa, direta ou indiretamente, por Recife.

Q4 — Árvore de busca em profundidade (DFS)

A partir de Recife, a função `dfs()` percorre todo o grafo e gera a árvore abaixo (indicando entre parênteses os tempos de descoberta e finalização de cada vértice):

None

Recife (desc=1, fim=52)

└ Olinda (desc=2, fim=3)

└ Paulista (desc=4, fim=5)

└ Camaragibe (desc=6, fim=7)

└ Jaboatao (desc=8, fim=9)

└ Caboatã (desc=10, fim=11)

└ Ipojuca (desc=12, fim=13)

└ Vitoria_Santo_Antao (desc=14, fim=51)

└ Palmares (desc=15, fim=50)

└ Garanhuns (desc=16, fim=49)

└ Caruaru (desc=17, fim=48)

└ Bezerros (desc=18, fim=19)

└ Belo_Jardim (desc=20, fim=43)

└ Pesqueira (desc=21, fim=42)


```
└─ Arcoverde (desc=22, fim=41)

    └─ Serra_Talhada (desc=23, fim=36)

        └─ Salgueiro (desc=24, fim=33)

            └─ Petrolina (desc=25, fim=30)

                └─ Araripina (desc=26, fim=29)

                    └─ Ouricuri (desc=27, fim=28)

                        └─ Floresta (desc=31, fim=32)

                            └─ Afogados (desc=34, fim=35)

                                └─ Buique (desc=37, fim=40)

                                    └─ Aguas_Belas (desc=38, fim=39)

                                        └─ Surubim (desc=44, fim=47)

                                            └─ Limoeiro (desc=45, fim=46)
```

Além da árvore, a DFS identificou 11 arestas de retorno: (Caruaru, Recife), (Bezerros, Vitoria_Santo_Antao), (Pesqueira, Caruaru), (Arcoverde, Caruaru), (Araripina, Salgueiro), (Ouricuri, Petrolina), (Floresta, Arcoverde), (Buique, Pesqueira), (Aguas_Belas, Garanhuns), (Surubim, Recife) e (Limoeiro, Recife).

Q5 — O grafo possui ciclos?

Sim. Em um grafo não dirigido, a DFS classifica uma aresta (u, v) como aresta de retorno quando v já foi descoberto, ainda não foi finalizado, e não é o pai direto de u

na árvore de busca (condição $\text{desc}[v] < \text{desc}[u]$ no código). Fisicamente, isso significa que já existe um caminho na árvore ligando u a v , e a aresta (u, v) fecha esse caminho, formando um ciclo. Como foram encontradas 11 arestas desse tipo (listadas na questão anterior), a malha rodoviária estudada possui pelo menos 11 ciclos — ou seja, ao menos 11 pares de cidades já conectados pela árvore de busca também possuem uma rota alternativa direta entre si. Do ponto de vista logístico, isso é positivo: existem rotas alternativas caso uma rodovia seja interditada.

Q6 — Qual a cidade mais isolada do Sertão na rede?

Para responder a essa pergunta, duas métricas foram combinadas: a distância acumulada a partir de Recife (via Dijkstra) e o grau do vértice (isolamento estrutural).

Por distância: Ouricuri é a cidade mais distante de Recife na rede, a 750 km, alcançada pelo caminho Recife → Caruaru → Arcoverde → Serra_Talhada → Salgueiro → Araripina → Ouricuri — 40 km a mais que Petrolina.

Por grau (estrutura): Afogados tem grau 1, conectando-se apenas a Serra_Talhada. É o único vértice de grau 1 localizado no Agreste/Sertão — os demais vértices de grau 1 (Olinda, Caboatã, Paulista, Camaragibe, Jaboatão e Ipojuca) ficam todos na Região Metropolitana do Recife, a poucos quilômetros da capital.

Ou seja, Ouricuri é a cidade "mais isolada" em termos de distância percorrida a partir da capital, enquanto Afogados é a mais isolada em termos de dependência de uma única rota: se a rodovia até Serra_Talhada for interditada, Afogados fica imediatamente desconectada do restante do grafo. Os dois resultados, juntos, reforçam que o Sertão depende de rotas mais longas e com menos alternativas do que a Região Metropolitana.

5. Conclusão

A análise da rede de municípios de Pernambuco utilizando o dataset D8 demonstrou a aplicação prática da teoria dos grafos na modelagem de redes de transporte. A representação por lista de adjacência mostrou-se eficiente para o grafo, composto por 26 vértices e 36 arestas.

Os algoritmos implementados (BFS, DFS, Dijkstra e identificação de componentes conexas) permitiram analisar a estrutura da rede, mostrando que o grafo é conexo, possui ciclos e tem Recife como o principal hub, com 10 conexões diretas. Além disso, o algoritmo de Dijkstra encontrou o menor caminho entre Recife e Petrolina, com distância total de 710 km.

Dessa forma, o trabalho atingiu seus objetivos, permitindo aplicar conceitos de teoria dos grafos e demonstrando sua importância na análise de redes de transporte e na resolução de problemas reais.

6. Referências

CORMEN, Thomas H. et al. **Algoritmos: teoria e prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

SEEDGEWICK, Robert; WAYNE, Kevin. **Algorithms**. 4. ed. Boston: Addison-Wesley, 2011.

PYTHON SOFTWARE FOUNDATION. The Python Standard Library: csv, heapq, collections. [S. l.]: Python Software Foundation, 2024. Disponível em: <https://docs.python.org/3/library/>. Acesso em: 01/07/2026.